

Assignment 6

Task 1 Symbol Table with primitive types

Theoretical Background

This task is based on the concepts from Lecture Slides 9–19.

In MicroJava, the **symbol table** stores information about all declared names in the program, including variables, methods, constants, and types. Each declared name is represented by an `Obj` node (Slide 9) that tracks:

- its name, kind (e.g., `Var`, `Meth`)
- its type, address, and declaration level

The compiler maintains nested scopes using:

- `Tab.openScope()` to open a new method/class scope (Slide 17)
- `Tab.closeScope()` to return to the enclosing scope (Slide 18)

Objects are inserted into the current scope using `Tab.insert()` (Slide 19), which assigns addresses to variables in order of declaration and detects name clashes.

MJ-code snippet

```
program P
final int a = 5;
int b,c;
{
    void x (int c, char d)
    {}

    void main (int f)
    int g;
    {}
}
```

We describe the state of the symbol table after parsing the opening brace `{` of each method.

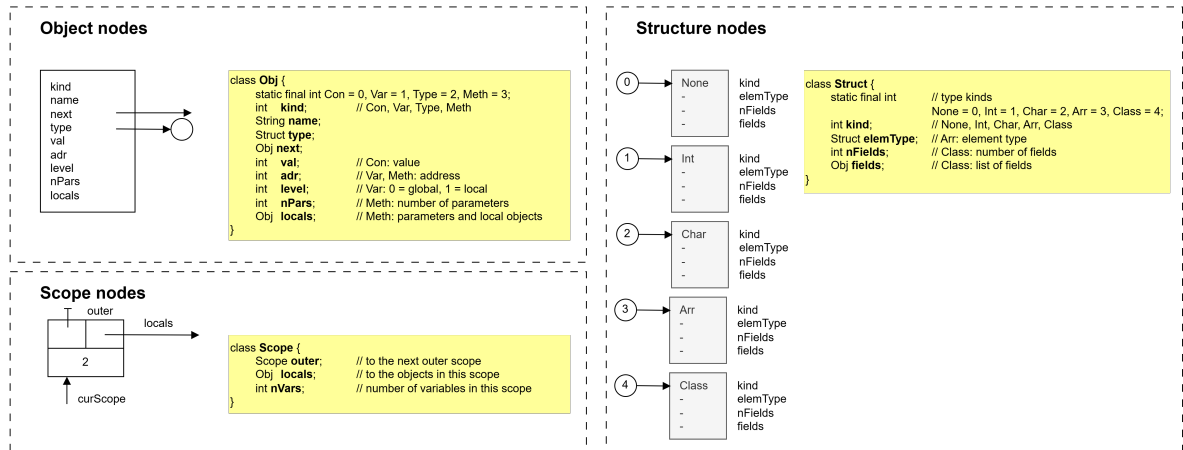
Understanding the Symbol Table Tables (Lecture Slides 9, 12, 19)

Each row in the symbol table tables represents an `Obj` entry (Lecture Slide 9). The columns mean:

- **Kind:** One of `Var`, `Meth`, `Type`, or `Con` — identifies what the object represents.
- **Name:** The identifier name in the source code.
- **Type:** The declared type of the object — e.g., `int`, `char`, `void`, `array`, etc.
- **Address:** For variables, it's the offset in memory. For methods, it can represent a code label.
- **Level:** Scope level — 0 for global scope, 1 for local scopes (e.g., method bodies).

These values are assigned when the compiler processes declarations via `Tab.insert(...)` (Lecture Slide 19).

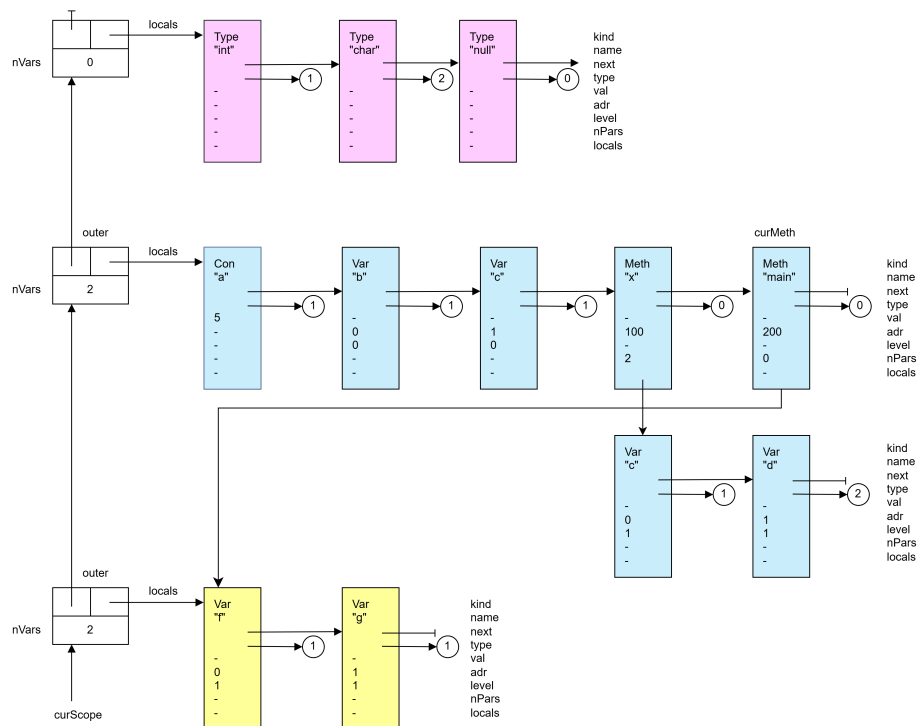
Solution



```

program P
    final int a = 5;
    int b, c;
    {
        void x (int c, char d)
        {}

        void main (int f)
        {
            int g;
            void main (int f)
        }
    }
    
```



Task 2 Symbol Table with Array

Theoretical Background

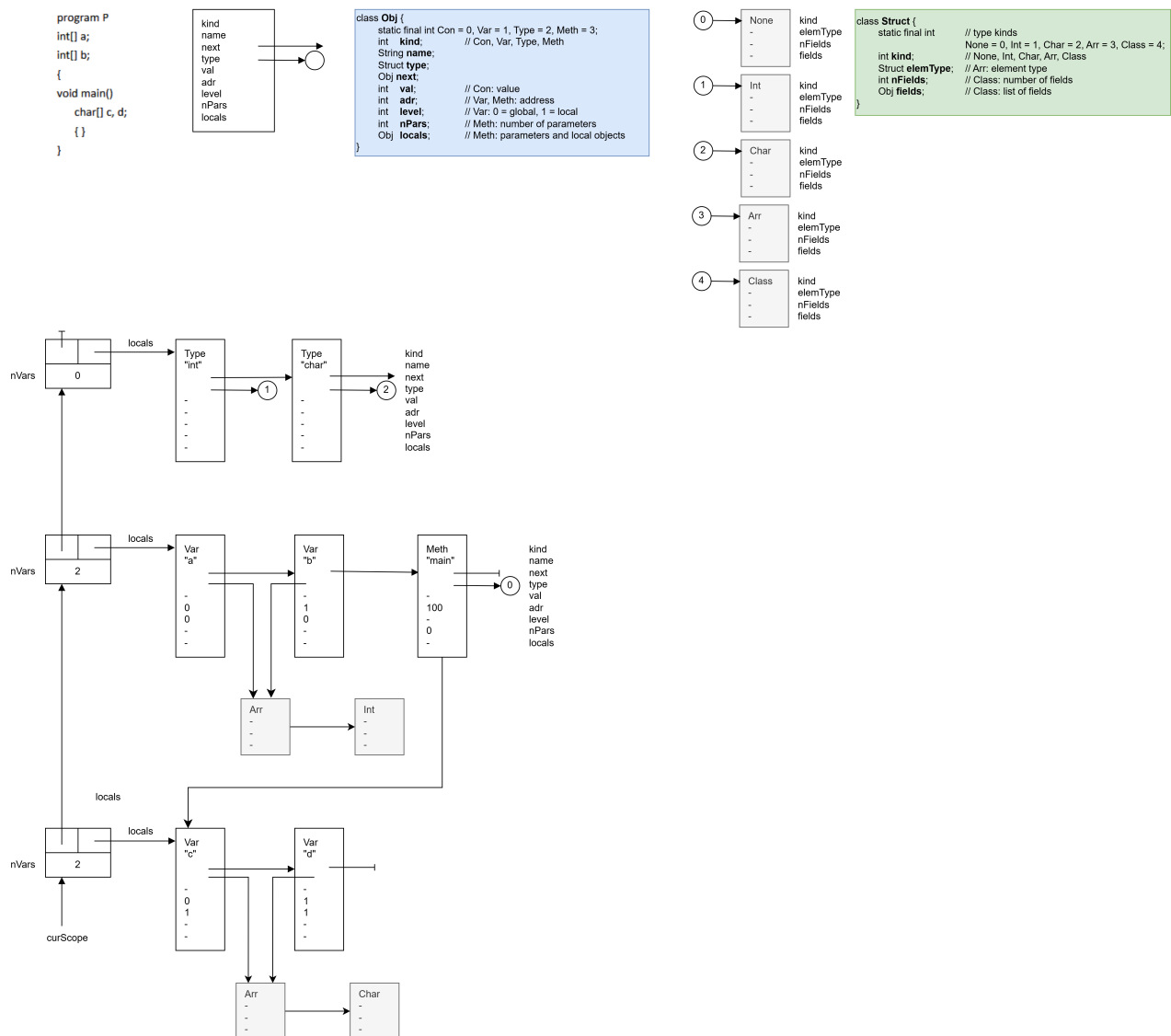
This task uses array types in MicroJava, based on Lecture Slides 9, 12, 17–19, and 30–32.

Arrays are reference types in MicroJava. Their types are stored in **Struct** nodes with:

- `kind = Arr`
- `elemType = type of array element (e.g., intType)`

Variables are inserted into the symbol table via `Tab.insert()`, which assigns addresses and levels depending on the current scope (`curScope`, Slide 19).

Solution



Task 3 Symbol Table with Classes

Theoretical Background

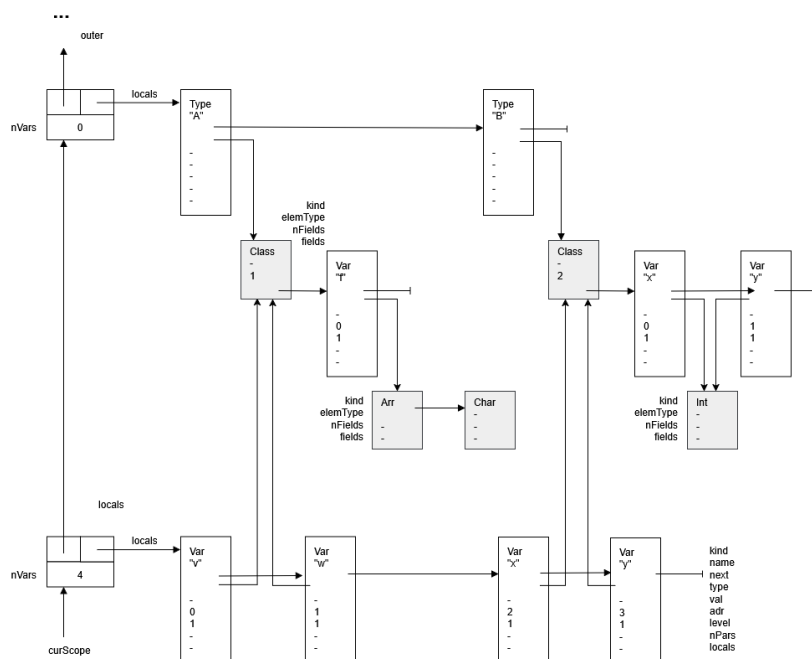
This task demonstrates the handling of class types in MicroJava. According to Lecture Slides 9 and 33:

- Class types are represented by Struct objects with `kind = Class`.
- Fields declared inside a class are inserted into the structure node's field list.
- A class name is inserted as an `Obj.Type` pointing to the structure.

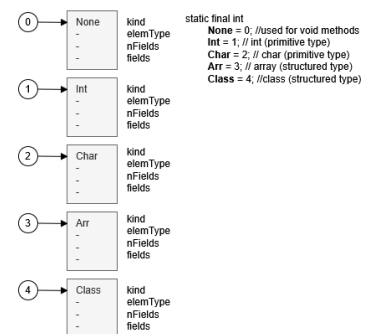
MJ-code snippet

```
program P
class A {
char[] f;
}
class B {
int x,y;
}
{
void main()
A v,w;
B x,y;
}
```

Solution



```
program P
class A { char[] f; }
class B { int x, y; }
{
void main()
A v, w; B x, y;
{
}
```



Task 4 Dumping the Symbol Tabl
